VECTORES EN LA INGENIERÍA DE SOFTWARE

Julian David Ortiz Espinosa

Jhonatan Smith Caviedes Rojas

Uniespinal

técnica Profesional en Programación Web

Rafael Arturo Suarez Velasco

6 de Abril de 2026

1.Sistemas de Coordenadas en el DOM vs. Cartesiano

Sistema de coordenadas cartesiano

Las coordenadas cartesianas se utilizan para representar gráficamente una relación matemática o movimiento en posición física. Además, se caracterizan por tener como referencia unos ejes ortogonales que concurren entre sí en el llamado punto de origen. En este sistema, el origen se ubica en el punto (0,0), y los ángulos se miden desde el eje X positivo en sentido antihorario. Este modelo es ampliamente utilizado en geometría, física y trigonometría.

estos se representan mediante dos ejes perpendiculares

- El eje X (horizontal), que crece hacia la derecha.
- El eje Y (vertical), que crece hacia arriba.

Sistema de coordenadas en el DOM

Las coordenadas en el DOM son valores numéricos en píxeles que indican la posición exacta de un elemento o el cursor dentro de la ventana del navegador. Se basan en un sistema (horizontal) e (vertical), donde el punto es la esquina superior izquierda, permitiendo manipular elementos con JavaScript.

¿Por qué en la web el eje Y crece hacia abajo?

La esquina superior izquierda de la pantalla es (0, 0), es decir, las coordenadas X e Y son (0, 0). Esto significa que el cero vertical corresponde al punto más alto y el cero horizontal al punto más a la izquierda. Así a medida que el sistema dibuja hacia abajo, el valor de Y aumenta. Esto debido a que Los monitores antiguos (CRT) dibujaban la pantalla secuencialmente de arriba hacia abajo, línea por línea. Esta convención se mantuvo en la programación gráfica.

¿esto como afecta al calculo de ángulos de dirección?

Como el eje Y está invertido El crecimiento hacia abajo hace que las direcciones verticales se inviertan. El Sistema Parece estar “volteado”.

Para calcular correctamente el ángulo en la web, se suele usar:

$$\theta = \arctan 2(-y, x)$$

En esta formula se debe colocar un signo negativo a la Y, ya que sin este se consigue un ángulo invertido. Así el menos corrige y muestra el ángulo esperado.

2. Vectores Unitarios y Normalización

Es importante la normalización en el movimiento diagonal de un videojuego 2d ya que por ejemplo si un personaje se mueve así:

- Derecha → $(1, 0)$
- Arriba → $(0, -1)$
- Diagonal (derecha + arriba) → $(1, -1)$

El vector resultante debería medir 1 pero si calculamos su magnitud esta es 1.41 esto resultando en que el jugador o enemigos se muevan mas rápido de manera diagonal, así afectando la mecánica de movimiento sintiéndose incomoda o injusta. Estos problemas se solucionan cambiando el vector a uno unitario, es decir, cambiar su longitud a uno pero manteniendo su dirección.

3.Producto Punto ($Dot Product$) en Detección de Colisiones

Producto punto o escalar

El producto escalar es una herramienta matemática sencilla pero extremadamente útil. Codifica la relación entre las magnitudes y direcciones de dos vectores en un único valor. Este valor es una forma de medir qué tan alineados están dos vectores.

Para calcularlo se debe multiplicar sus componentes en X, y sus componentes en Y para después sumarlas:

$$\vec{A} \cdot \vec{B} = A_x B_x + A_y B_y$$

Determinar posición del objeto

El producto escalar es una herramienta clave en juegos 2D y 3D porque permite comparar direcciones entre vectores. Con él puedes saber si algo está al frente, detrás o si dos objetos se están acercando. Para esto se necesitan dos vectores de por ejemplo, el jugador y el enemigo, aquí se calcula el producto punto para saber que tan alineados están estos dos vectores. Si ambos vectores apuntan más o menos en la misma dirección, el resultado del producto escalar es positivo. Eso significa que el enemigo está delante del personaje. Si apuntan en direcciones opuestas, el resultado es negativo, lo que indica que el enemigo está detrás. Y si el resultado es cercano a cero, significa que el enemigo está hacia un lado, ni delante ni detrás.

- < 0 → se están acercando (posible colisión)
- > 0 → se están alejando
- = 0 → movimiento perpendicular

Colisión inminente

Para saber si hay colisión inminente se compara el vector de posición (de A hacia B) con el vector de movimiento relativo.

- Si el resultado indica que ambos vectores apuntan en direcciones opuestas (valor negativo), entonces la distancia entre los objetos está disminuyendo significa que se están acercando por lo que puede haber colisión.
- Si indica que apuntan en la misma dirección (valor positivo), la distancia aumenta por lo que se están alejando.
- Si es cercano a cero, el movimiento es lateral por lo que no hay acercamiento directo.

4.Interpolación Lineal (Lerp):

es una función matemática utilizada en programación y gráficos para calcular un valor intermedio entre dos puntos (A y B). este se usa para mover objetos de una posición inicial a una final de manera progresiva.

Formula $P = P_0 + t(P_1 - P_0)$

Aquí, P_0es el valor inicial, P_1es el valor final y $t$es un parámetro que varía entre 0 y 1. permite pasar de un valor a otro de forma progresiva y continua. esta expresión toma la diferencia entre el punto final y el inicial ($P1-P0$) que representa “cuánto hay que moverse”, y luego multiplica ese cambio por t, que indica qué porcentaje del recorrido se lleva. Finalmente, se lo suma al punto inicial.

Por eso:

- cuando t=0, no te has movido entonces estás en $P0$
- cuando t=1, ya hiciste todo el recorrido entonces estás en P1
- cuando t está entre 0 y 1, estás en algún punto intermedio

JavaScript

En JavaScript, se utiliza directamente. Se define una función que aplica la fórmula y luego se hace que el valor de $t$aumente con el tiempo. A medida que $t$crece, el objeto se mueve suavemente desde su posición inicial hasta la final. Esto suele hacerse con “requestAnimationFrame”, que actualiza la animación cuadro por cuadro. El resultado es un movimiento fluido porque cada frame representa un pequeño avance.

CSS

En CSS, no escribes la fórmula explícitamente, pero el navegador hace internamente algo equivalente cuando usas propiedades como transition o animation. Por ejemplo, si defines una transición de transform: translateX(0px) a translateX(100px), el motor del navegador calcula posiciones intermedias entre esos dos valores en cada instante de tiempo. Es decir, está aplicando el mismo principio de interpolación.

Bibliography

ayuware. (n.d.). *ayuware.es/blog/para-que-nos-sirven-las-coordenadas-cartesianas/#que-son-las-coordenadas-cartesianas*. Retrieved from ayuware.es: https://www.ayuware.es/blog/para-que-nos-sirven-las-coordenadas-cartesianas/#que-son-las-coordenadas-cartesianas

Azmi, A. (n.d.). *amirazmi.net/dot-products-in-games-and-their-use-cases/*. Retrieved from amirazmi.net: https://amirazmi.net/dot-products-in-games-and-their-use-cases/

DevMark, G. L. (2024, may 11). Retrieved from youtube.com: https://youtu.be/BApEahU_eFI?si=g9BgO4_I63uu_-dZ

Garg, N. (2023, Dec 28). Retrieved from https://www.geeksforgeeks.org/: https://www-geeksforgeeks-org.translate.goog/javascript/javascript-coordinates-of-mouse/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=sge&_x_tr_hist=true

Indierama. (2023, mar 17). Retrieved from youtube.com: https://youtu.be/tNJLlDqyRXw?si=6hKjbVn-2QpoX7XB

Indierama. (2023, mar 17). Retrieved from youtube.com: https://youtu.be/tNJLlDqyRXw?si=jx0vPIaZhnvpIEV-